REPLACE AND MODERNIZE LEGACY VPN ARCHITECTURES WITH A CLOUD-NATIVE ZSCALER ZERO TRUST SOLUTION TO REDUCE ATTACK SURFACE AND LATERAL MOVEMENT.

Project Details

Title:Replace and modernize legacy VPN architectures with a cloud-native Zscaler Zero Trust solution to reduce attack surface and lateral movement.

In today's digital environment, organizations require secure remote access for employees and users to connect to internal applications and resources. Traditionally, Virtual Private Networks (VPNs) have been used to provide this access by creating encrypted connections between users and the organization's network. However, many legacy VPN architectures follow a perimeter-based security model where users who successfully authenticate are given broad access to the internal network. This approach increases security risks, as attackers who gain access can move laterally within the network and potentially reach sensitive systems.

With the growth of cloud services and remote work, traditional VPN solutions are becoming less effective in protecting modern IT environments. To address these challenges, organizations are adopting the **Zero Trust security model**, which follows the principle of **"never trust, always verify."** Instead of providing full network access, Zero Trust continuously verifies user identity and allows access only to specific applications.

This project focuses on replacing traditional VPN infrastructure with a **cloud-native Zscaler Zero Trust solution**. By providing secure, identity-based access to applications rather than the entire network, the system reduces the attack surface and prevents unauthorized lateral movement. This approach improves overall security while environments

Abstract:

In modern organizations, secure remote access to internal applications and data has become an important requirement due to the growth of cloud computing and remote work environments. Traditionally, Virtual Private Networks (VPNs) have been used to provide remote connectivity. However, many legacy VPN architectures follow a perimeter-based security approach that allows users broad access to internal networks after authentication. This increases security risks, as attackers who gain access through compromised credentials or devices can move laterally within the network and potentially access sensitive resources.

To address these challenges, organizations are adopting the **Zero Trust security model**, which is based on the principle of "never trust, always verify." Instead of trusting users simply because they are inside the network, Zero Trust continuously verifies user identity, device security, and access permissions before granting access to applications.

This project focuses on replacing traditional VPN infrastructure with a **cloud-native Zero Trust solution using Zscaler**. The Zscaler platform provides secure and direct access to applications without exposing the internal network to the internet. By connecting users only to the applications they are authorized to use, the system significantly reduces the attack surface and prevents unauthorized lateral movement.

The proposed approach enhances network security, improves access control, and supports modern cloud-based environments. Overall, implementing a Zscaler Zero Trust architecture helps organizations strengthen cybersecurity, protect sensitive data, and provide secure remote access in a scalable and efficient manner.

Use Case Scenarios

Scenario 1: Remote Employee Access to Internal Application

An employee working from home needs access to the organization's internal HR application. Instead of connecting through a traditional VPN, the user connects using the Zscaler Zero Trust solution. The system verifies the user's identity and device security before granting access. The user is only allowed to connect to the HR application and cannot access other internal resources.

Output:

- User identity verified (Employee_User_Group)
- Secure authentication through identity provider
- Connection restricted to HR Application only
- Internal network remains hidden
- No lateral movement to other systems possible

Scenario 2: Vendor Access to Internal System

A third-party vendor requires temporary access to a specific internal system to perform maintenance tasks. Using the Zscaler Zero Trust model, the vendor is provided limited access only to the required application rather than the entire network.

Output:

- User identity verified (Vendor_User_Group)
- Time-limited access granted
- Connection restricted to maintenance system
- Network segmentation maintained
- Unauthorized resources remain inaccessible

Scenario 3: Student Access to Research Server

A research student needs to access a university server to run data analysis scripts. Through Zscaler Zero Trust, the student can securely connect to the research server without exposing the entire university network.

Output:

- User identity verified (Research_Student_Group)
- Secure micro-tunnel connection established
- Access limited to research server only
- Other network services blocked
- Secure encrypted communication maintained

Scenario 4: IT Administrator Access to Servers

An IT administrator needs to manage internal servers for maintenance and monitoring. With Zero Trust architecture, the administrator can access specific servers securely without exposing management ports to the internet.

Output:

- User identity verified (IT_Admin_Group)
- Privileged access policy enforced
- Secure connection to specific server
- Activity monitoring and logging enabled
- No access to unrelated systems

Scenario 5: Guest User Access to Library Resources

A visiting faculty member requires temporary access to digital library resources hosted on the internal server. The Zero Trust solution allows access only to the library application without connecting the user to the entire network.

Output:

- User identity verified (Guest_Faculty_Group)
- Temporary session access provided
- Connection restricted to Library Server
- Network segmentation maintained
- Sensitive systems remain protected

Stage 1

Title of the project:

Replace and modernize legacy VPN architectures with a cloud-native Zscaler Zero Trust solution

Overview:

Traditional Virtual Private Network (VPN) architectures were designed to provide secure remote access to internal corporate networks. However, these legacy systems often grant users broad network access once connected, which increases the risk of unauthorized access, lateral movement, and potential data breaches. With the rapid adoption of cloud computing, remote work, and mobile devices, traditional VPN solutions are no longer sufficient to secure modern IT environments.

To overcome these limitations, organizations are adopting the **Zero Trust Architecture**, which follows the principle of "never trust, always verify." In this model, every user, device, and application must be authenticated and authorized before access is granted. Instead of allowing full network access like traditional VPNs, Zero Trust provides secure and controlled access only to specific applications.

A cloud-native platform such as the **Zscaler Zero Trust Exchange**, developed by **Zscaler**, helps organizations replace legacy VPN architectures with a more secure and scalable solution. It connects users directly to applications without exposing the internal network and continuously verifies user identity, device posture, and security policies. This approach reduces the attack surface and improves overall security.

By modernizing legacy VPN infrastructure with a Zero Trust solution, organizations can enhance data protection, support secure remote access, and improve user experience. It also simplifies network management and provides better scalability for cloud-based applications and distributed work environments.

List of teammates:

S.NO	NAME	COLLEGE	CONTACT
1	Varishma S	AVP College of Arts and Science	rajisrvm@gmail.com
2	Gayathri S	AVP College of Arts and Science	gayathrisivakumar786@gmail.com
3	Divya Dharshini p	AVP College of Arts and Science	poochappan2@gmail.com
4	Muthu Dharshana Priya p	AVP College of Arts and Science	darshanapriya2005@gmail.com

List of Vulnerability Table:

S.NO	CATEGORY NAME	VULNERABILITY NAME	CWE/CVE - NO
1	Broken Access Control	Relative Path Traversal	CWE-23
2	Security Misconfiguration	Password in Configuration File	CWE-260
3	Software Supply Chain Failures	Unimplemented or Unsupported Feature in UI	CWE-447
4	Cryptographic Failures	Key Exchange without Entity Authentication	CWE-322

5	Injection	Improper Neutralization of HTTP Headers for Scripting Syntax	CWE-644
6	Insecure Design	Trust Boundary Violation	CWE-501
7	Authentication Failures	Origin Validation Error	CWE-346
8	Software or Data Integrity Failures	Untrusted Search Path	CWE-426
9	Security Logging & Alerting Failures	Information Loss or Omission	CWE-221
10	Mishandling of Exceptional Conditions	Divide By Zero	CWE-369

REPORT:

1. **Vulnerability name:** Relative Path Traversal

CWE/CVE: 200

OWASP/SANS category: OWASP Top 10 – Broken Access Control

Description:

Relative Path Traversal is a vulnerability that occurs when an application allows users to access files and directories outside the intended folder structure on the server. This happens when user input is not properly validated and attackers manipulate file paths using special characters such as to move up the directory hierarchy.

By exploiting this vulnerability, attackers can gain access to sensitive files stored on the server that should normally be restricted. These files may include system configuration files, application source code, password files, or other confidential data.

Path traversal attacks take advantage of improper input validation in file handling functions. When applications directly use user input to construct file paths without proper filtering or security checks, attackers can modify the path to retrieve unauthorized files.

This vulnerability can lead to serious security risks such as unauthorized data access, exposure of confidential information, and potential system compromise.

Vulnerability path

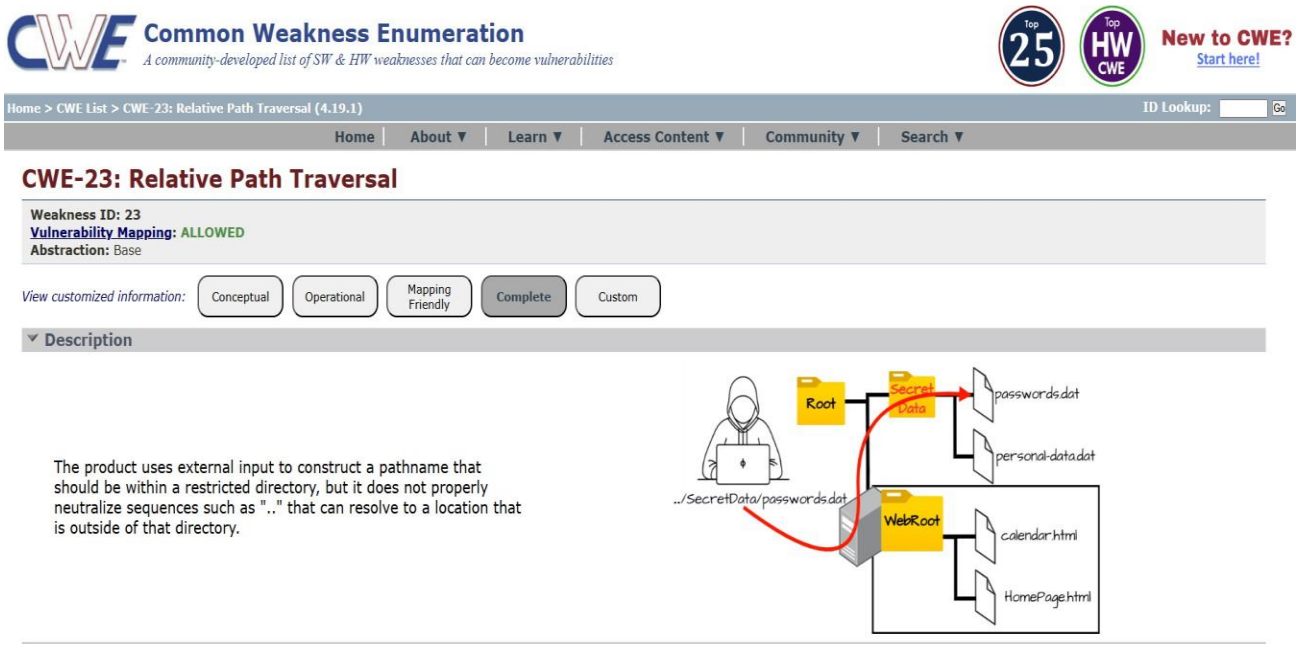
http://example.com/download?file=../../etc/passwd

Business impact

Relative Path Traversal can expose sensitive files stored on the server, including system configuration files, application source code, and confidential data. Attackers who gain access to these files may obtain important system information that helps them plan further attacks.

This vulnerability may lead to information disclosure, unauthorized access, and system compromise**.** Attackers may use the exposed information to identify additional vulnerabilities, bypass security controls, or gain deeper access to the system.

For organizations, this vulnerability can result in data breaches, financial losses, operational disruptions, and damage to organizational reputation**.** If confidential data or internal system files are exposed, it may also lead to compliance violations and loss of customer trust.



Recommendation/Mitigation

• Validate and sanitize all user inputs before using them in file path operations.

• Restrict file access to specific directories and prevent users from navigating outside the intended folder structure.

• Implement proper input validation and filtering to block directory traversal sequences such as “../”.

• Use secure file handling methods and avoid directly using user input to construct file paths.

• Apply the principle of least privilege so that applications only have access to the files they require.

• Conduct regular security testing and code reviews to identify and fix path traversal vulnerabilities.

2. Vulnerability Name: Password in Configuration File

CWE/CVE: CWE-260

OWASP/SANS Category: OWASP Top 10- Security Misconfiguration

Description

Password in Configuration File vulnerability occurs when sensitive credentials such as passwords, API keys, or database access details are stored in plain text within application configuration files. These configuration files are often used to store important system settings required for the application to function.

If these files are not properly secured, attackers may gain access to them through misconfigured permissions, source code exposure, backup files, or server misconfigurations. When passwords are stored in plain text, anyone who gains access to the configuration file can easily read and use the credentials.

Attackers can use these exposed credentials to access databases, internal services, or administrative systems connected to the application. Storing passwords in configuration files without proper protection increases the risk of unauthorized access and compromise of sensitive systems.This vulnerability often occurs due to poor security practices during development or improper configuration management.

Vulnerability Path

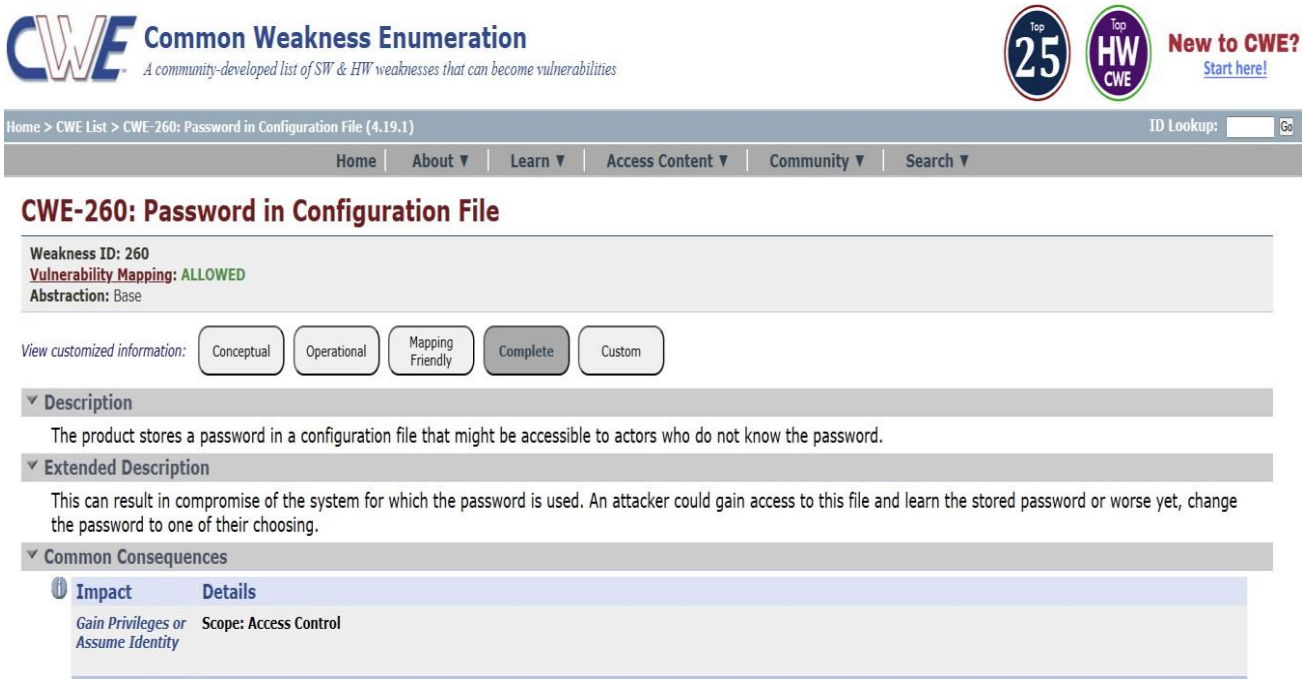
http://example.com/config/application.properties

Business Impact

Storing passwords in configuration files can expose sensitive credentials that are critical for accessing internal systems such as databases, APIs, or administrative services. If attackers obtain these credentials, they may gain unauthorized access to important resources within the organization.

This vulnerability may lead to unauthorized system access, data breaches, and compromise of confidential information. Attackers could manipulate or delete data, disrupt system operations, or gain deeper access to internal networks.

For organizations, this can result in financial losses, operational disruption, regulatory compliance issues, and reputational damage. Exposure of credentials may also affect customer trust if sensitive data is accessed or misused by unauthorized parties.



Recommendation / Mitigation

• Avoid storing passwords or sensitive credentials in plain text within configuration files.

• Use secure credential storage mechanisms such as environment variables, encrypted configuration files, or secret management systems.

• Implement strong access controls to restrict who can access configuration files.

• Encrypt sensitive data stored in configuration files whenever possible.

• Regularly rotate passwords and credentials used by applications and services.

• Conduct regular security audits and configuration reviews to identify and remove exposed credentials.

3. **Vulnerability Name:** Unimplemented or Unsupported Feature in UI

CWE/CVE: CWE-447

OWASP/SANS Category: OWASP Top 10- Software Supply Chain Failures

Description:

Unimplemented or Unsupported Feature in UI vulnerability occurs when an application's user interface includes features or options that are not fully implemented, properly supported, or securely handled in the backend system. These features may appear in the interface as buttons, menu options, or links that developers planned to implement but did not complete before deployment.

If such features remain accessible in the user interface, attackers may attempt to interact with them to discover hidden functionality or unexpected system behavior. In some cases, these incomplete features may expose internal APIs, debugging functions, or test components that were not intended for public use.

Attackers may exploit these unsupported or partially implemented features to bypass normal application controls, access restricted resources, or trigger unexpected system responses. This vulnerability often occurs due to incomplete development processes, poor testing, or lack of proper code review before deployment.Leaving unimplemented features in the user interface increases the risk of exposing hidden system functionality and may introduce additional security weaknesses.

Vulnerability Path

http://example.com/feature-test

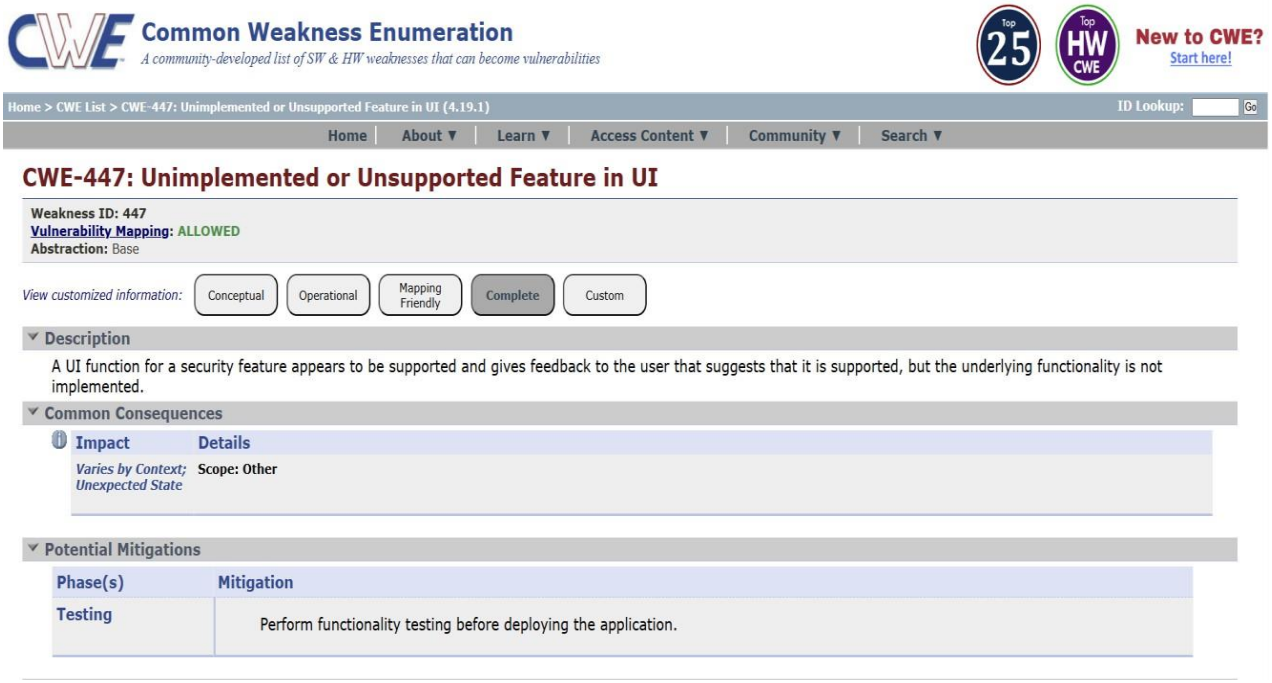
Business Impact

Unimplemented or unsupported features in the user interface can expose hidden system functions that were not intended for public access. Attackers may use these features to explore application behavior and identify potential security weaknesses.

This vulnerability may lead to unauthorized access, exposure of internal system functions, or unexpected application behavior. Attackers might exploit these features to bypass security mechanisms or access sensitive resources within the system.

For organizations, such vulnerabilities can result in system instability, security breaches, and operational disruptions. If attackers discover hidden or experimental functionality, it may also expose sensitive information or introduce additional security risks.

Additionally, the presence of incomplete or unsupported features may reduce user trust and indicate weaknesses in the organization's development and testing processes.



Common Weakness Enumeration

A community-developed list of SW & HW weaknesses that can become vulnerabilities

Top 25 | Top HW CWE | New to CWE? Start here!

Home > CWE List > CWE-447: Unimplemented or Unsupported Feature in UI (4.19.1) | ID Lookup: Go

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-447: Unimplemented or Unsupported Feature in UI

Weakness ID: 447
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

A UI function for a security feature appears to be supported and gives feedback to the user that suggests that it is supported, but the underlying functionality is not implemented.

Common Consequences

Impact	Details
Varies by Context; Unexpected State	Scope: Other

Potential Mitigations

Phase(s)	Mitigation
Testing	Perform functionality testing before deploying the application.

Recommendation / Mitigation

• Remove all unimplemented or unsupported features from the user interface before deploying the application.

• Ensure that incomplete or experimental functionality is disabled or restricted in production environments.

• Conduct thorough code reviews and security testing to identify unused or unsupported components.

• Implement proper access control mechanisms to prevent unauthorized interaction with hidden system features.

• Follow secure software development lifecycle (SDLC) practices to ensure that only fully tested and supported features are deployed.

• Regularly perform security assessments and vulnerability scans to detect unintended functionality in the application

4. **Vulnerability Name:** Key Exchange without Entity Authentication

CWE/CVE: CWE-322

OWASP/SANS Category: OWASP Top 10 – Cryptographic Failures

Description:

This vulnerability occurs when a cryptographic key exchange process is performed without properly verifying the identity of the communicating parties. In secure communication systems, key exchange mechanisms are used to establish encryption keys between two entities so that sensitive data can be transmitted securely. However, if the system does not authenticate the entities involved in the key exchange, attackers may intercept the communication and impersonate one of the parties.

Without entity authentication, an attacker can perform a man-in-the-middle attack and establish separate encryption keys with both parties while secretly monitoring or modifying the transmitted data. As a result, confidential information may be exposed or altered without the knowledge of the legitimate users. This weakness undermines the security of the cryptographic process and reduces the effectiveness of encryption mechanism.

Vulnerability Path:

http://example.com/api/secure-communication

Business Impact:

Cryptographic failures caused by improper key exchange mechanisms can lead to serious security risks for organizations. If attackers intercept the communication during the key exchange process, they may gain access to sensitive data such as login credentials, personal information, or financial transactions.

This vulnerability may allow attackers to eavesdrop on confidential communications, modify transmitted data, or impersonate legitimate users within the system. Such attacks can compromise the integrity and confidentiality of sensitive information and may disrupt secure business operations.

Vulnerability Path:

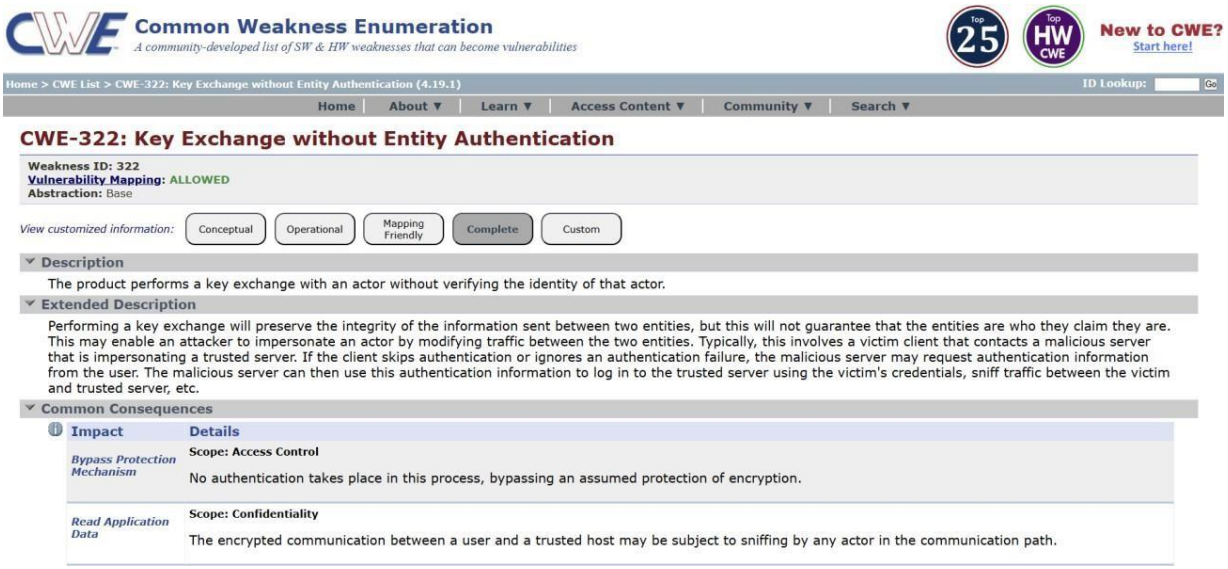
http://example.com/api/secure-communication

Business Impact:

Cryptographic failures caused by improper key exchange mechanisms can lead to serious security risks for organizations. If attackers intercept the communication during the key exchange process, they may gain access to sensitive data such as login credentials, personal information, or financial transactions.

This vulnerability may allow attackers to eavesdrop on confidential communications, modify transmitted data, or impersonate legitimate users within the system. Such attacks can compromise the integrity and confidentiality of sensitive information and may disrupt secure business operations.

For organizations, these security breaches can result in financial losses, reputational damage, and loss of customer trust. Exposure of confidential data may also lead to regulatory penalties and legal consequences, especially when organizations fail to protect sensitive user information properly.



Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities

Top 25 | Top HW CWE | New to CWE? Start here!

Home > CWE List > CWE-322: Key Exchange without Entity Authentication (4.19.1) | ID Lookup:

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-322: Key Exchange without Entity Authentication

Weakness ID: 322
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product performs a key exchange with an actor without verifying the identity of that actor.

Extended Description

Performing a key exchange will preserve the integrity of the information sent between two entities, but this will not guarantee that the entities are who they claim they are. This may enable an attacker to impersonate an actor by modifying traffic between the two entities. Typically, this involves a victim client that contacts a malicious server that is impersonating a trusted server. If the client skips authentication or ignores an authentication failure, the malicious server may request authentication information from the user. The malicious server can then use this authentication information to log in to the trusted server using the victim's credentials, sniff traffic between the victim and trusted server, etc.

Common Consequences

Impact	Details
Bypass Protection Mechanism	**Scope: Access Control** No authentication takes place in this process, bypassing an assumed protection of encryption.
Read Application Data	**Scope: Confidentiality** The encrypted communication between a user and a trusted host may be subject to sniffing by any actor in the communication path.

Recommendation / Mitigation:

• Implement secure key exchange protocols that include proper entity authentication to verify the identity of both communicating parties.

• Use strong and well-established cryptographic protocols such as TLS with certificate-based authentication to protect communication channels.

• Ensure that digital certificates are properly validated before establishing secure connections between systems.

• Avoid using weak or outdated cryptographic algorithms that may expose the system to security risks.

• Regularly review and update cryptographic implementations to follow modern security standards and best practices.

• Conduct regular security testing and code reviews to identify weaknesses in cryptographic processes and improve overall system security.

5. **Vulnerability Name:** Improper Neutralization of HTTP Headers for Scripting Syntax

CWE/CVE: CWE-644

OWASP/SANS Category: OWASP Top 10 – Injection

Description

This vulnerability occurs when a web application fails to properly validate or sanitize HTTP header inputs before processing them. HTTP headers such as User-Agent, Referer, or Host may contain malicious scripting syntax inserted by an attacker. If the application processes these headers without neutralizing the malicious content, the injected script may be executed within the application environment or reflected in responses sent to other users. Attackers can exploit this weakness to inject malicious scripts or commands through manipulated HTTP headers. This may lead to unauthorized actions, execution of malicious code, or manipulation of application

behavior. Improper validation of header inputs therefore creates an opportunity for attackers to exploit injection vulnerabilities and compromise application security.

Vulnerability Path:

http://example.com/page

Business Impact:

Injection vulnerabilities related to improper neutralization of HTTP headers can create serious security risks for web applications. When attackers are able to insert malicious scripting syntax into HTTP header fields, the application may process this data without proper validation or filtering. As a result, malicious scripts may be executed within the application environment or reflected in responses sent to users.

Attackers can exploit this vulnerability to manipulate application behavior, steal session information, or gain unauthorized access to sensitive user data. In some cases, injected scripts may allow attackers to hijack user sessions, perform unauthorized actions on behalf of users, or modify the content displayed by the application.

For organizations, such attacks may lead to data breaches, service disruption, and compromise of confidential information. Exposure of sensitive data can damage the organization's reputation and reduce customer trust. In addition, security incidents caused by injection vulnerabilities may result in financial losses and potential legal or regulatory consequences.

Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-644: Improper Neutralization of HTTP Headers for Scripting Syntax (4.19.1) ID Lookup: Go

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-644: Improper Neutralization of HTTP Headers for Scripting Syntax

Weakness ID: 644
Vulnerability Mapping: ALLOWED
Abstraction: Variant

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product does not neutralize or incorrectly neutralizes web scripting syntax in HTTP headers that can be used by web browser components that can process raw headers, such as Flash.

Extended Description

An attacker may be able to conduct cross-site scripting and other attacks against users who have these components enabled.

If a product does not neutralize user controlled data being placed in the header of an HTTP response coming from the server, the header may contain a script that will get executed in the client's browser context, potentially resulting in a cross site scripting vulnerability or possibly an HTTP response splitting attack. It is important to carefully control data that is being placed both in HTTP response header and in the HTTP response body to ensure that no scripting syntax is present, taking various encodings into account.

Common Consequences

Impact	Details
Execute Unauthorized Code or Commands	**Scope: Integrity, Confidentiality, Availability** Run arbitrary code.
Read Application Data	**Scope: Confidentiality** Attackers may be able to obtain sensitive information.

Recommendation / Mitigation:

• All HTTP header inputs should be properly validated and sanitized before being processed by the application.

• Applications should implement strict input validation mechanisms to detect and block malicious scripting syntax in request headers.

• Output encoding should be applied when displaying header values in web pages to prevent execution of injected scripts.

• Security libraries and frameworks should be used to automatically handle input validation and reduce the risk of injection vulnerabilities.

• Web application firewalls should be deployed to detect and block malicious requests containing header-based injection attempts.

• Regular security testing and vulnerability scanning should be conducted to identify and fix injection weaknesses within the application.

6. Vulnerability Name: Trust Boundary Violation

CWE/CVE: CWE-501

OWASP/SANS Category: OWASP Top 10 – Insecure Design

Description:

This vulnerability occurs when an application incorrectly trusts data that crosses a security boundary without performing proper validation or verification. In secure system design, a trust boundary separates components with different levels of trust, such as user input, external systems, or internal services. When data moves from a less trusted area to a more trusted compone nt without adequate validation, it can lead to a trust boundary violation. Attackers can exploit this weakness by sending malicious or manipulated data to the application, which may then be processed as if it were trustworthy. As a result, the system may perform unauthorized actions, expose sensitive information, or allow attackers to bypass security controls.

Vulnerability Path:

http://example.com/api/userdata

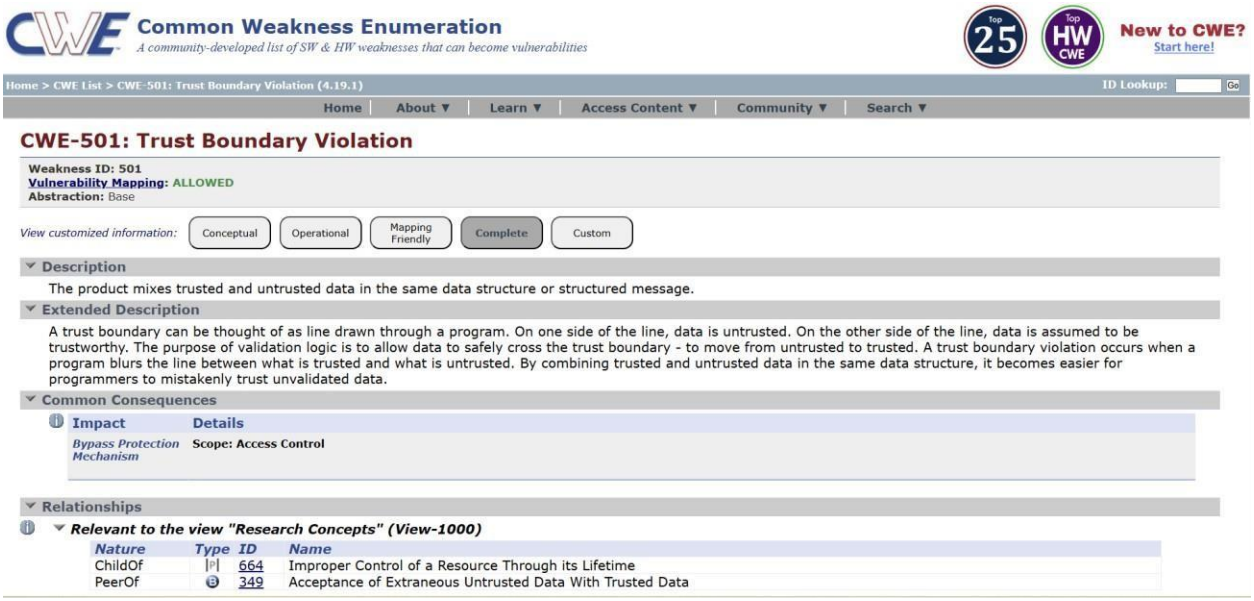
Business Impact:

Trust boundary violations can lead to serious security risks within an application because untrusted data may be processed without proper validation. Attackers can exploit this weakness to manipulate system behavior, bypass security checks, or gain unauthorized access to sensitive resources. When the application incorrectly trusts external inputs or third-party systems, malicious data may enter critical components of the system.

This vulnerability can allow attackers to modify application logic, perform unauthorized transactions, or access confidential information stored within the system. In some cases, attackers may escalate privileges or interfere with internal system operations.

For organizations, such security weaknesses can result in data breaches, service disruption, and financial losses. Exposure of sensitive information may damage the organization's reputation and

reduce customer confidence in the system. In addition, organizations may face legal or regulatory consequences if they fail to properly protect sensitive user or business data.



Recommendation / Mitigation:

• Applications should clearly define trust boundaries and ensure that all data crossing these boundaries is properly validated and sanitized.

• Strong input validation mechanisms should be implemented to verify data received from users, external systems, or third-party services.

• Security controls should be applied at every layer of the application to prevent untrusted data from affecting critical system components.

• Developers should follow secure design principles and perform threat modeling during the system design phase.

• Access control mechanisms should be enforced to ensure that only authorized entities can interact with sensitive system components.

• Regular security testing and design reviews should be conducted to identify and fix weaknesses related to trust boundary violations.

7. **Vulnerability Name:** Origin Validation Error

CWE/CVE: CWE-346

OWASP/SANS Category: OWASP Top 10 – Authentication Failures

Description:

Origin Validation Error occurs when a web application fails to properly verify the origin of requests sent to the server. Web applications often rely on origin headers, referrer headers, or other mechanisms to determine whether a request is coming from a trusted source. If this validation is not correctly implemented, attackers may be able to send unauthorized requests to the application.

When origin validation is weak or missing, malicious websites or attackers can trick authenticated users into sending requests to the vulnerable application without their knowledge. This can allow attackers to perform actions on behalf of the user, such as modifying account settings, submitting forms, or accessing restricted resources.

Attackers may exploit this vulnerability by crafting malicious requests or using specially designed web pages to send unauthorized requests to the application. Without proper origin verification, the server may treat these requests as legitimate.This vulnerability often occurs due to improper implementation of security checks or reliance on untrusted request headers without proper validation.

Vulnerability Path

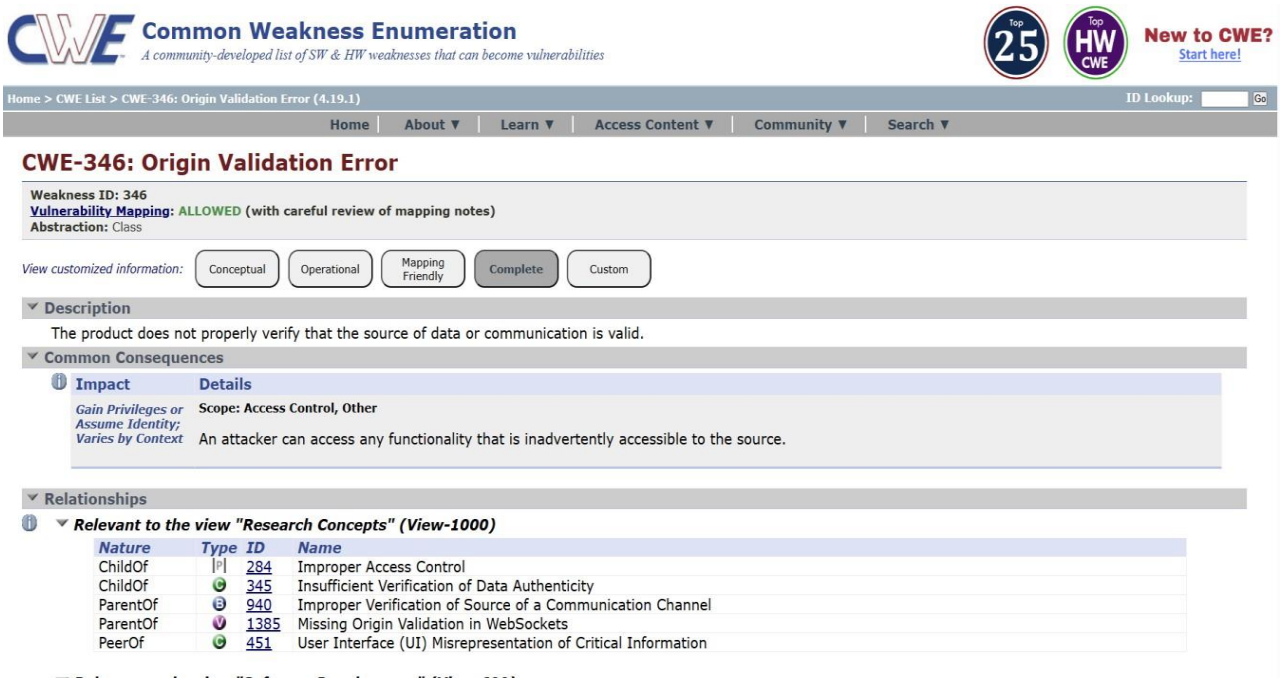
http://example.com/api/update-profile

Business Impact

Origin validation errors can allow attackers to send unauthorized requests to the application while impersonating legitimate users. If attackers successfully exploit this vulnerability, they may perform actions without the user's consent.

This vulnerability may lead to unauthorized account modifications, data manipulation, or unauthorized access to protected resources. Attackers could change account settings, submit unauthorized transactions, or manipulate application data.

For organizations, this may result in data integrity issues, financial losses, security breaches, and reputational damage. If attackers exploit user accounts or perform unauthorized actions, it can also reduce customer trust and lead to potential compliance issues.



Recommendation / Mitigation

• Implement proper origin and referrer validation to ensure requests come from trusted sources.

• Use CSRF protection mechanisms such as anti-CSRF tokens to verify legitimate user requests.

• Avoid relying solely on client-side checks or untrusted HTTP headers for security validation.

• Configure the application to verify trusted domains and request sources before processing sensitive actions.

• Perform regular security testing and vulnerability assessments to detect origin validation issues.

• Apply secure authentication and session management practices to prevent unauthorized request execution.

8. Vulnerability Name: Untrusted Search Path

CWE/CVE: CWE-426

OWASP/SANS Category: OWASP Top 10 – Software or Data Integrity Failures

Description:

Untrusted Search Path vulnerability occurs when an application loads executable files, libraries, or system resources from directories that are not properly controlled or trusted. In many operating systems, applications search multiple directories to locate required files such as executable programs or dynamic libraries.

If the application searches directories that are writable by untrusted users or attackers, a malicious file with the same name as the expected file can be placed in those directories. When the application attempts to load the file, it may execute the attacker's malicious file instead of the legitimate one.

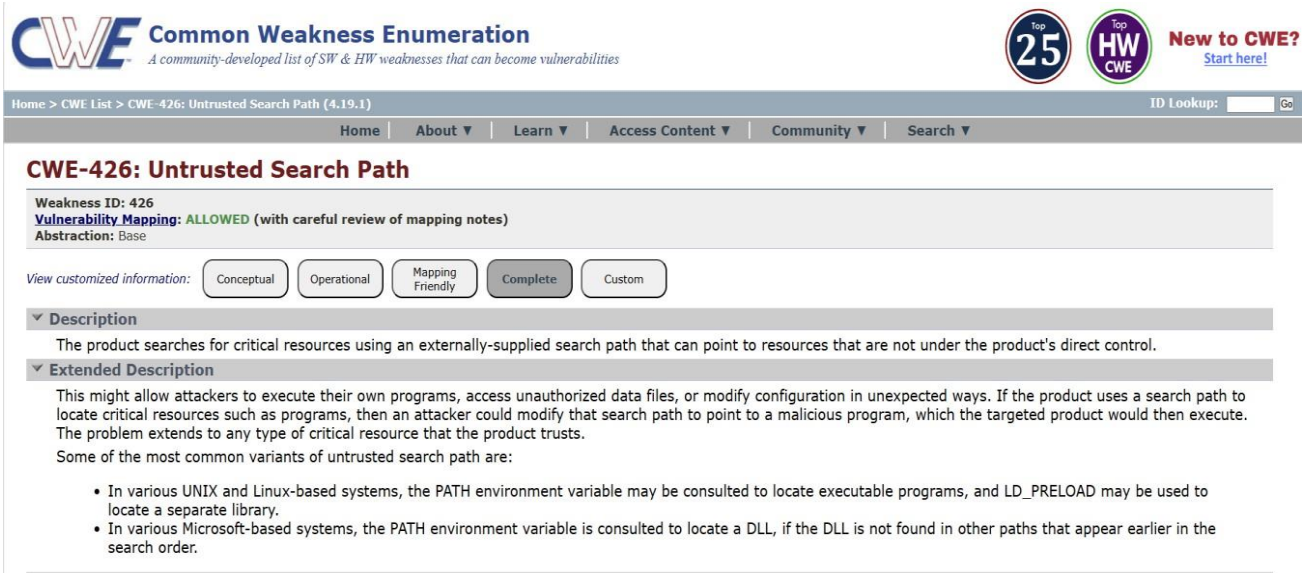
This vulnerability typically occurs due to improper configuration of system paths, insecure file loading mechanisms, or failure to specify the exact location of required executables or libraries.Attackers can exploit this vulnerability by placing malicious files in directories that appear earlier in the search path. As a result, the application may unknowingly execute the malicious file, leading to unauthorized code execution.

Vulnerability Path

C:\Program Files\App\run.exe

Business Impact

Untrusted Search Path vulnerabilities can allow attackers to execute malicious code within the application or system environment. If attackers successfully place malicious files in a directory that is searched by the application, the system may execute the malicious program instead of the legitimate one. This vulnerability may lead to unauthorized code execution, system compromise, or privilege escalation. Attackers could manipulate application behavior, install malware, or gain deeper access to the system. For organizations, this can result in system disruption, data compromise, financial loss, and reputational damage. If critical applications are affected, it may also lead to operational downtime and security incidents.



Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-426: Untrusted Search Path (4.19.1)

ID Lookup:

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-426: Untrusted Search Path

Weakness ID: 426
Vulnerability Mapping: ALLOWED (with careful review of mapping notes)
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product searches for critical resources using an externally-supplied search path that can point to resources that are not under the product's direct control.

Extended Description

This might allow attackers to execute their own programs, access unauthorized data files, or modify configuration in unexpected ways. If the product uses a search path to locate critical resources such as programs, then an attacker could modify that search path to point to a malicious program, which the targeted product would then execute. The problem extends to any type of critical resource that the product trusts.

Some of the most common variants of untrusted search path are:

- In various UNIX and Linux-based systems, the PATH environment variable may be consulted to locate executable programs, and LD_PRELOAD may be used to locate a separate library.
- In various Microsoft-based systems, the PATH environment variable is consulted to locate a DLL, if the DLL is not found in other paths that appear earlier in the search order.

Recommendation / Mitigation

• Always specify absolute paths when loading executables, libraries, or system resources.

• Avoid loading files from directories that are writable by untrusted users.

• Implement proper access control and file permission settings to restrict unauthorized modifications.

• Validate and verify the integrity of executable files and libraries before loading them.

• Follow secure coding practices to ensure that applications only load trusted components.

• Conduct regular security reviews and vulnerability assessments to identify insecure search path configurations.

9. **Vulnerability Name:** Information Loss or Omission

CWE/CVE: CWE-221

OWASP/SANS Category: OWASP Top 10 – Security Logging and Monitoring Failures

Description:

This vulnerability occurs when an application fails to properly record important security events or omits critical information from system logs. Logging and monitoring mechanisms are essential for detecting suspicious activities, identifying security incidents, and supporting forensic investigations. If important events such as login attempts, system errors, data access, or configuration changes are not recorded correctly, security teams may not be able to detect attacks or investigate incidents effectively. Information loss or omission in logs can occur due to improper logging configuration, incomplete log generation, or failure to capture essential system activities. As a result, attackers may perform malicious actions within the system without being detected.

Vulnerability Path:

http://example.com/login

Business Impact:

Security logging and monitoring failures can significantly reduce an organization's ability to detect and respond to cyberattacks. When important system activities are not properly logged, attackers may exploit vulnerabilities, access sensitive data, or perform unauthorized actions without leaving sufficient evidence in the system logs.

This vulnerability can make it difficult for administrators to identify the source of security incidents, track malicious activities, or determine how an attack occurred. As a result, organizations may experience delayed responses to security breaches and increased damage to their systems and data.

For organizations, insufficient logging and monitoring can lead to data breaches, operational disruptions, and financial losses. In addition, failure to maintain proper security logs may violate regulatory or compliance requirements. This can damage the organization's reputation and reduce customer trust in the system's ability to protect sensitive information.

CWE-221: Information Loss or Omission

Weakness ID: 221
Vulnerability Mapping: ALLOWED (with careful review of mapping notes)
Abstraction: Class

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product does not record, or improperly records, security-relevant information that leads to an incorrect decision or hampers later analysis.

Common Consequences

Impact	Details
Hide Activities	**Scope: Non-Repudiation**

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
ChildOf	P	664	Improper Control of a Resource Through its Lifetime
ParentOf	B	222	Truncation of Security-relevant Information
ParentOf	B	223	Omission of Security-relevant Information
ParentOf	B	224	Obscured Security-relevant Information by Alternate Name
ParentOf	B	356	Product UI does not Warn User of Unsafe Actions
ParentOf	B	396	Declaration of Catch for Generic Exception
ParentOf	B	397	Declaration of Throws for Generic Exception

Recommendation / Mitigation:

• Applications should implement comprehensive logging mechanisms to record all important security events and system activities.

• Logs should include detailed information such as timestamps, user identifiers, IP addresses, and the type of activity performed.

• Security monitoring tools should be used to analyze logs and generate alerts for suspicious or abnormal behavior.

• Log files should be securely stored and protected from unauthorized modification or deletion.

• Regular log reviews and audits should be conducted to detect potential security incidents and investigate abnormal activities.

• Organizations should implement centralized logging systems to ensure consistent monitoring across all applications and infrastructure components.

10. Vulnerability Name: Divide By Zero

CWE-369

OWASP/SANS Category: Mishandling of Exceptional Conditions

Description

Divide By Zero vulnerability occurs when a program attempts to divide a number by zero without properly handling the condition. In many programming languages, dividing by zero results in a runtime error or system exception that may cause the application to crash or behave unexpectedly.

This vulnerability usually occurs when user input or system data is used in mathematical operations without proper validation. If an attacker is able to control or influence the input value and set it to zero, the application may attempt to perform a division operation using that value.

Improper handling of such exceptions can lead to system instability, application crashes, or denial of service conditions. In some cases, error messages generated by the system may also expose sensitive debugging or system information.

Failing to handle divide-by-zero conditions indicates poor input validation and lack of proper exception handling in the application.

Vulnerability Path

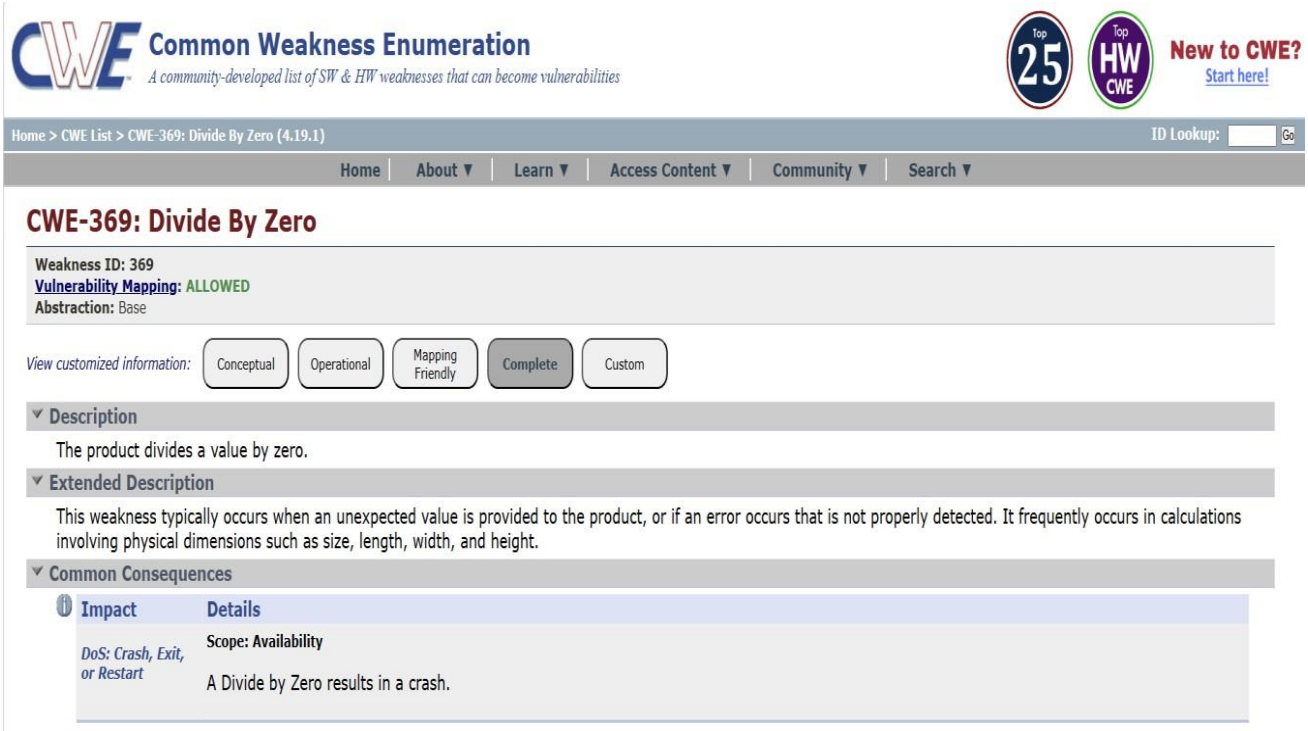
http://example.com/calculate?value=0

Business Impact

Divide By Zero vulnerabilities can cause application crashes or unexpected behavior when mathematical operations are performed with invalid input values. Attackers may intentionally provide zero values to trigger errors in the system.

This vulnerability may lead to application failure, denial of service, or interruption of system functionality. If the application stops functioning properly, legitimate users may not be able to access services or perform important tasks.

For organizations, this can result in service downtime, operational disruption, and reduced system reliability. Repeated exploitation of such errors may also affect user experience and damage the organization's reputation.



Recommendation / Mitigation

• Validate all user inputs before performing mathematical operations.

- Implement proper error handling and exception management to prevent application crashes.
- Ensure that division operations include checks to verify that the denominator is not zero.
- Use secure coding practices to handle unexpected or invalid input values.
- Conduct regular code reviews and application testing to identify and fix logical errors in the system.
- Implement logging and monitoring to detect abnormal input patterns that may trigger system errors.

Stage 2

Overview:

Nessus is a widely used vulnerability assessment tool designed to identify security weaknesses in computer systems, networks, and applications. It is developed by Tenable and is commonly used by security professionals, network administrators, and organizations to improve system security. The main purpose of Nessus is to scan systems and detect vulnerabilities that could potentially be exploited by attackers.

Nessus works by performing automated vulnerability scans on target systems. During the scanning process, the tool sends various network requests to the target machine and analyzes the responses received. By examining these responses, Nessus can collect information about the operating system, running services, open ports, installed applications, and system configurations. Based on this information, the scanner determines whether the system contains any known vulnerabilities.

One of the key features of Nessus is its plugin-based architecture. Plugins are small programs that are designed to detect specific security issues. Each plugin checks for a particular vulnerability such as outdated software versions, missing security patches, weak passwords, insecure configurations, or exposed network services. Nessus contains thousands of plugins, and these plugins are continuously updated to detect newly discovered vulnerabilities.

The scanning process usually begins with host discovery, where the tool identifies active systems on the network. After discovering the target systems, Nessus performs port scanning to determine which ports and services are open. Once the services are identified, the scanner runs appropriate plugins to check those services for potential vulnerabilities. This automated process helps detect security weaknesses efficiently without manually checking each system.

After the scanning process is completed, Nessus generates a detailed vulnerability report. The report lists all detected vulnerabilities along with important information such as the vulnerability name, severity level, description, possible impact, and recommended solutions.

Vulnerabilities are usually categorized into different severity levels including Critical, High, Medium, Low, and Informational. This classification helps administrators prioritize which issues should be fixed first.

Overall, Nessus plays an important role in cybersecurity by helping organizations identify weaknesses in their systems before attackers can exploit them. Regular vulnerability scanning using Nessus helps improve system security, reduce risks, and maintain a safer network environment.

Target website : https://demo.testfire.net/

Target ip address: 65.61.137.117

List of vulnerability

s.no	**Vulnerability name**	**Severity**	**plugins**
1	SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)	LOW	83875
2	TLS Version 1.0 Protocol Detection	MEDIUM	104743
3	HTTP Server Type and Version	INFO	10107

4	Open Port Re-check	INFO	10919
5	SSL Cipher Suites Supported	INFO	21643
6	Apache Tomcat Detection	INFO	39446
7	Nessus SYN scanner	INFO	11219
8	Additional DNS Hostnames	INFO	46180
9	Device Type	INFO	54615
10	OS Identification	INFO	11936

REPORT

1. Vulnerability Name: SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)

Severity : LOW

Plugin: 83875

Port : 443

Description:

The remote host allows SSL/TLS connections with one or more Diffie-Hellman moduli less than or equal to 1024 bits. Through cryptanalysis, a third party may be able to find the shared secret in a short amount of time (depending on modulus size and attacker resources). This may allow an attacker to recover the plaintext or potentially violate the integrity of connections.

Solution: Reconfigure the service to use a unique Diffie-Hellman moduli of 2048 bits or greater.

Business Impact:

The presence of weak SSL/TLS Diffie-Hellman keys (≤1024 bits) can reduce the strength of encrypted communications between the server and its users. Attackers with sufficient computing resources may potentially break the encryption and gain access to sensitive information transmitted over the network. This could lead to data exposure, loss of confidentiality, and unauthorized interception of communications.

For organizations, this vulnerability may result in reputational damage, reduced customer trust, and possible non-compliance with modern security standards and regulatory requirements. Although the severity is classified as low, leaving weak cryptographic configurations in place can still weaken the overall security posture and make the system more vulnerable to advanced cryptographic attacks.

2. Vulnerability Name: TLS Version 1.0 Protocol Detection

Severity : MEDIUM

Plugin: 104743

Port : 443

Description:

The remote service accepts connections encrypted using TLS 1.0. TLS 1.0 has a number of cryptographic design flaws. Modern implementations of TLS 1.0 mitigate these problems, but newer versions of TLS like 1.2 and 1.3 are designed against these flaws and should be used.

As of March 31, 2020, Endpoints that aren't enabled for TLS 1.2 and higher will no longer function properly with major web browsers and major vendors. PCI DSS v3.2 requires that TLS 1.0 be disabled entirely by June 30, 2018, except for POS POI terminals (and the SSL/TLS termination points to which they connect) that can be verified as not being susceptible to any known exploits.

Solution:

Enable support for TLS 1.2 and 1.3, and disable support for TLS 1.0.

Business Impact:

The presence of TLS 1.0 on the server indicates that the system supports an outdated and deprecated encryption protocol. Modern security standards such as Payment Card Industry Security Standards Council (PCI SSC) and guidelines from National Institute of Standards and Technology (NIST) recommend disabling TLS 1.0 because it is considered cryptographically weak.

Support for TLS 1.0, an outdated and deprecated cryptographic protocol, may expose sensitive data transmitted over HTTPS (port 443) to interception or decryption by attackers exploiting known weaknesses. This could potentially lead to unauthorized access to confidential information, resulting in data breaches, reputational damage, and loss of customer trust. Additionally, the use of insecure protocols may lead to non-compliance with industry security standards such as PCI DSS, which could result in regulatory penalties or audit findings

3. Vulnerability Name: HTTP Server Type and Version

Severity : INFO

Plugin: 10107

Port :80

Description:

This test gives some information about the remote HTTP protocol - the version used, whether HTTP Keep-Alive is enabled, etc... This test is informational only and does not denote any security problem.

Business Impact:

The disclosure of the web server type and version through HTTP responses may provide attackers with valuable information about the underlying technology stack. By identifying the exact server software and version, attackers can more easily search for known vulnerabilities associated with that specific version and attempt targeted exploitation.

Although this issue does not directly compromise the system, it increases the likelihood of targeted attacks and reconnaissance activities. For organizations, this may lead to an elevated risk of exploitation if unpatched vulnerabilities exist, potentially affecting the confidentiality, integrity, or availability of the web application and its supporting infrastructure.

4. Vulnerability Name: Open Port Re-check

severity: INFO

Plugin: 10919

Port : N/A

Description:

One of several ports that were previously open are now closed or unresponse The scan may have caused a service to freeze or stop running. An administrator may have stopped a particular service during the scanning process. This might be an availability problem related to the following :A network outage has been experienced during the scan, and the remote network cannot be reached anymore by the scanner. This scanner may has been blacklisted by the system administrator or by an automatic intrusion detection / prevention system that detected the scan. The remote host is now down, either because a user turned it off during the scan or because a select denial of service was effective. In any case, the audit of the remote host might be incomplete and may need to be done again.

Solution: Steps to resolve this issue include :
increase checks_read_timeout and/or reduce max_checks. Disable any IPS during the Nessus scan

Business Impact:

The presence of open ports on a system may increase the attack surface by exposing network services that could potentially be discovered and targeted by attackers. If unnecessary or insecure services are running on these open ports, attackers may attempt to exploit known vulnerabilities associated with those services to gain unauthorized access or disrupt system operations.

Although this finding is categorized as informational, improperly managed or unnecessary open ports may lead to security risks such as unauthorized service access, reconnaissance opportunities for attackers, and potential exploitation of vulnerable services, which could impact the confidentiality, integrity, or availability of organizational systems and data.

5. Vulnerability Name: SSL Cipher Suites Supported

severity : INFO

Plugin: 21643

Port : 443

Description:

The scan detected that the HTTPS service supports multiple SSL Cipher Suites, which are combinations of cryptographic algorithms used to secure communications over Transport Layer Security (TLS) connections. The plugin reports the list of cipher suites that the server is willing to negotiate during the SSL/TLS handshake process.

Some cipher suites may provide stronger encryption and security than others. If weak or outdated cipher suites are enabled, attackers may attempt to exploit them to weaken the encryption protecting the communication channel.

Business Impact:

The availability of certain SSL/TLS cipher suites may expose the server to potential security risks if weak or outdated encryption algorithms are supported. Attackers could attempt to

exploit weaker cipher suites to compromise the confidentiality or integrity of sensitive data transmitted between users and the server.

Although this finding is categorized as informational, the presence of weak or less secure cipher suites may increase the risk of cryptographic attacks, potentially leading to data interception, reduced trust in secure communications, and non-compliance with modern security best practices.

6. Vulnerability Name: Apache Tomcat Detection

severity : INFO

Plugin: 39446

Port: 80

Description:

The scan detected that the web server is running Apache Tomcat, an open-source web server and servlet container commonly used to deploy Java-based web applications. This detection is typically based on server responses, banners, or identifiable characteristics exposed by the application service.

Identifying the presence of Apache Tomcat allows security tools and administrators to determine the underlying application platform and assess whether the installed version is up to date and properly configured. If version or configuration details are exposed, attackers may use this information during reconnaissance to identify potential vulnerabilities associated with the detected software.

Business Impact:

The detection of Apache Tomcat indicates that the web application is running on a specific server platform. Disclosure of the underlying server technology may assist attackers during reconnaissance by helping them identify potential vulnerabilities, misconfigurations, or known security issues associated with that platform.

Although this finding is informational, publicly exposed technology details may increase the risk of targeted attacks if the server is not regularly patched or securely configured. This could

potentially lead to exploitation attempts that may affect the confidentiality, integrity, or availability of the web application and associated organizational data.

7. Vulnerability Name: Nessus SYN scanner

severity : INFO

Plugin: 11219

Port : 80

Description:

The scan detected that the host responded to a TCP SYN Scan performed by the Nessus vulnerability scanner. A SYN scan is a common network reconnaissance technique used to identify open ports and determine whether services are actively listening on a target system.

This plugin reports that the scanner successfully performed a SYN-based port scan to identify accessible services on the target host.

solution: Protect your target with an IP filter.

Business Impact:

The detection of open and responsive ports during a SYN scan indicates that network services are accessible from the scanning source. While this is a normal behavior for service discovery during security assessments, exposed services may also be identified by unauthorized attackers performing similar reconnaissance activities.

If unnecessary or poorly secured services are exposed, attackers may attempt to exploit vulnerabilities in those services, potentially leading to unauthorized access, service disruption, or compromise of sensitive organizational data.

8. Vulnerability Name: Additional DNS Hostnames

severity :INFO

Plugin: 46180

Port : N/A

Description:

Hostnames different from the current hostname have been collected by miscellaneous plugins. Nessus has generated a list of hostnames that point to the remote host. Note that these are only the alternate hostnames for vhosts discovered on a web server.Different web servers may be hosted on name-based virtual hosts.

Business Impact:

The disclosure of additional DNS hostnames may provide attackers with further insight into the organization's network structure and associated domains. These hostnames could expose additional applications, services, or environments that might not have been intended for public discovery.

Although this finding is informational, attackers may use this information for reconnaissance to identify additional targets for scanning or exploitation, potentially increasing the organization's overall attack surface and risk of unauthorized access.

9. Vulnerability Name: Device Type

severity : INFO

Plugin: 54615

Port : N/A

Description:

The scan identified the type of device responding on the network, such as a server, router, firewall, or other networked hardware. This identification is based on network responses, banners, or protocol fingerprints collected during the assessment. Knowing the device type helps in understanding the infrastructure and the roles of various systems within the environment.

Business Impact:

Disclosure of device type may aid attackers in planning targeted attacks by providing insight into the network architecture and specific hardware in use. While this information alone does not directly compromise the system, it can be leveraged in reconnaissance to identify

potentially vulnerable devices or services, increasing the risk of unauthorized access, exploitation, or disruption of critical organizational systems.

0. Vulnerability Name: OS Identification

severity: INFO

Plugin:11936

Port :N/A

Description:

The scan detected the operating system (OS) running on the target host. This information is typically determined through network fingerprinting techniques, analyzing responses from protocols and services to identify the OS type and version. Knowing the OS helps assess compatibility, configuration, and potential vulnerabilities associated with that platform.

Business Impact:

Disclosure of the target host's operating system may assist attackers in tailoring attacks to exploit OS-specific vulnerabilities or misconfigurations. While this finding is informational, it increases the effectiveness of reconnaissance efforts, potentially leading to targeted attacks, unauthorized access, or system compromise if vulnerabilities are present.

The identification of the target host's operating system (OS) provides attackers with critical insight into the environment, including the type, version, and potential patch level. With this information, attackers can focus on known vulnerabilities specific to that OS, increasing the likelihood of successful exploitation. It also enables more advanced reconnaissance by revealing likely installed services, default configurations, and potential security controls that could be bypassed.

Additionally, understanding the OS helps attackers map the network, identify high-value targets, and plan tailored attacks against critical systems. From a compliance perspective, exposing OS details may be considered a security misconfiguration, leading to potential audit findings or regulatory issues. While this finding is informational, it effectively increases the organization's attack surface and the risk of unauthorized access, data compromise, and disruption of system availability

STAGE 3

Report

TITLE:- Replace and modernize legacy VPN architectures with a cloud-native Zscaler Zero Trust solution to reduce attack surface and lateral movement

- Legacy VPN Architecture

Legacy VPN architecture is a network security model that creates an encrypted tunnel between a remote user and an organization's internal network using on-premises hardware appliances like VPN concentrators . Unlike modern solutions, legacy VPNs operate on an "authenticate once, trust indefinitely" principle. Once a user successfully authenticates, they are placed directly onto the internal network with an IP address, effectively becoming a node on that network .

This architecture was designed for a different era when all users were in offices and all applications were in data centers. It relies on the concept of a "trusted internal network" versus an "untrusted external internet"—the castle-and-moat model . The VPN concentrator acts as the drawbridge: once crossed, the user is trusted and granted broad access to internal resources based on their IP address rather than their identity .

- Limitations of Legacy VPN

Legacy VPN architectures present significant security and operational challenges in today's distributed work environment. The limitations include:

1. Over-permissioned Access: Once connected, users gain network-level access, meaning a compromised faculty laptop can scan, discover, and attack other systems on the internal network (lateral movement) . 2. Expanded Attack Surface: VPN concentrators must be exposed to the public internet with open ports (typically IPsec on UDP 500 or SSL VPN on TCP 443), making them visible targets for port scanning, brute-force attacks, and vulnerability exploitation .
3. Poor User Experience: Traffic is often "hair-pinned"—routed from the user to the data center and then out to the internet for SaaS applications—causing latency and slow performance .
4. Scalability Challenges: Hardware VPN appliances have hard capacity limits. During peak times (e.g., start of semester), users face connection failures, requiring manual upgrades of expensive hardware .
5. Operational Overhead: IT staff must patch, upgrade, and maintain physical VPN appliances, applying firmware updates and managing certificates .

- The Problem: Lateral Movement and Attack Surface

The core security weaknesses of legacy VPNs are the expanded attack surface and the risk of lateral movement.

Attack Surface: The VPN concentrator's publicly exposed IP address and open ports invite constant scanning

and attacks. Threat actors use tools to identify VPN appliances and exploit known vulnerabilities (e.g., CVE-2019-11510, a critical Pulse Secure VPN vulnerability). Each exposed port is a potential entry point .

Lateral Movement: Once an attacker gains access via a compromised credential or vulnerability, they inherit the user's network-level access. The internal network, traditionally flat, allows the attacker to move laterally. From a single compromised faculty laptop, they can:

- Scan for other vulnerable servers (e.g., databases, file shares)
- Perform ARP spoofing or pass-the-hash attacks

• Zero Trust Architecture

Zero Trust Architecture (ZTA) is a security framework that eliminates the concept of trusted and untrusted networks. It operates on the principle of "never trust, always verify" and "least privilege access." Every access request is fully authenticated, authorized, and encrypted before granting access, regardless of whether the user is in the office or working remotely .

The core tenets of Zero Trust include:

1. Assume Breach: Design the network as if attackers are already present.
2. Verify Explicitly: Authenticate and authorize based on identity, device health, and context.
3. Least Privilege Access: Grant only the minimum access necessary to perform a job function.
4. Micro-segmentation: Isolate workloads and users from one another to prevent lateral movement.

• Zscaler Zero Trust Solution

Zscaler is a cloud-native security platform that delivers Zero Trust capabilities entirely from the cloud, eliminating the need for on-premises VPN hardware. The solution consists of two primary components relevant to this project:

1. Zscaler Private Access (ZPA): A ZTNA service that provides seamless, identity-based access to private applications without placing users on the network. It connects users to applications, not the network .
2. Zscaler Internet Access (ZIA): A secure web gateway that inspects all internet-bound traffic for threats and data loss, protecting users regardless of location .

Together, they create a comprehensive Zero Trust replacement for legacy VPN architectures.

• How Zscaler Eliminates Attack Surface

Zscaler eliminates the attack surface by making the organization's applications completely invisible to the public internet.

This is achieved through:

1. No Inbound Ports: Zscaler ZPA uses Zscaler App Connectors—lightweight VMs deployed in the data center—that initiate *outbound* connections to the Zscaler cloud. No inbound firewall ports need to be opened for remote access .
2. Application Invisibility: To an external port scanner, the university's applications do not exist. There are no IP addresses to scan, no ports to probe, and no VPN concentrators to attack .
3. DNS Resolution: ZPA does not rely on public DNS to resolve internal application names. Applications are never exposed to the public DNS infrastructure .
4. Dark Cloud Principle: The Zscaler cloud acts as a broker. Users connect to the cloud, and the cloud connects to the application. The application server never sees the user’s IP, and the user never sees the server’s IP—only the application itself .

• How Zscaler Prevents Lateral Movement

Zscaler prevents lateral movement by ensuring users are connected to applications, not to the network. The mechanisms include:

1. Application Segmentation: ZPA establishes a one-to-one, encrypted micro-tunnel between the user's device and the specific application they are authorized to access. The user never receives an IP address on the internal network .

2. No Network Visibility: Because the user is not placed on the network, they cannot scan, discover, or connect to other resources. There is no routing to adjacent systems. The network is invisible .
3. Identity-Based Micro-segmentation: Access policies are tied to user identity, not IP addresses. Even if an attacker compromises a user's credentials, they can only access the specific applications that user is authorized for—nothing else .
4. Continuous Monitoring: Zscaler monitors the connection throughout the session. If suspicious behavior is detected (e.g., impossible travel, anomalous data transfer), the connection can be terminated in real-time, preventing further movement .

• Zscaler Architecture Components

To understand how Zscaler replaces the legacy VPN, it is essential to understand its core components:

1. Zscaler Client Connector: A lightweight software client installed on user devices (laptops, desktops, mobile devices). It intelligently forwards traffic:
   - Private application traffic → ZPA for secure access
   - Internet-bound traffic → ZIA for security inspection
   - Bypassed traffic → Direct to internet (for performance)
2. Zscaler App Connector: A software component deployed as a VM in the university's data center or cloud environment. It initiates outbound connections to the Zscaler cloud and brokers connections to internal applications. Key features:
   - No inbound firewall rules required
   - Automatically scales horizontally based on load
   - Can be deployed in multiple locations for high availability
3. Zscaler Cloud: The global security platform that enforces policies, brokers connections, and inspects traffic. It consists of over 150 data centers worldwide, ensuring low-latency access .

4. Zscaler Admin Portal: The centralized management console where administrators configure:
   - Access policies
   - Application definitions
   - User and group mappings
   - Security settings
   - Integration Points

Zscaler integrates with existing university infrastructure to leverage current investments:

1. Identity Provider (IdP) Integration:
   - Azure AD / Okta: Zscaler integrates with the university's existing IdP for user authentication and MFA. This ensures consistent identity management and allows users to use their existing credentials .
   - SAML 2.0 / OpenID Connect: Supports modern authentication protocols for seamless single sign-on (SSO).
2. Endpoint Protection Integration:
   - CrowdStrike / Microsoft Defender / Tanium: Zscaler checks device posture with integrated endpoint detection and response (EDR) tools. Access can be conditional on:

o Antivirus running and up-to-date o Disk encryption enabled o OS patch level current o Device not jailbroken/rooted

3. SIEM/SOAR Integration:
   - Splunk / Sentinel: Zscaler logs can be forwarded to the university's security information and event management (SIEM) system for centralized monitoring, alerting, and threat hunting .
4. Threat Intelligence Integration:
   - MISP Integration: Zscaler can consume threat intelligence feeds from MISP to block access to known malicious domains, IPs, and URLs via ZIA .

SOC and SIEM Implementation using Threat Intelligence

Security Operations Center (SOC):

A Security Operations Center (SOC) is a centralized team that monitors, detects, analyzes, and responds to cybersecurity incidents in an organization. The main goal of SOC is to protect the organization's systems, networks, and data from cyber threats.

The SOC team works continuously to identify suspicious activities, investigate alerts, and respond to attacks before they cause damage. It uses different security tools such as SIEM platforms, threat intelligence feeds, and endpoint protection systems to monitor the entire network.

SOC teams usually include security analysts, incident responders, threat hunters, and SOC managers who work together to ensure the organization's security.

SOC Cycle

The SOC operates in a continuous cycle to ensure proper security monitoring.

1. Monitoring

Security tools continuously monitor network traffic, user activities, and system logs to detect suspicious behavior.

2. Detection

Security alerts are generated when abnormal activities or potential threats are identified.

3. Analysis

SOC analysts investigate alerts to determine whether they are real threats or false positives.

4. Response

If a threat is confirmed, the SOC team takes action to stop the attack, isolate affected systems, and prevent further damage.

5. Recovery

Systems are restored and secured after the attack is contained.

6. Improvement

The SOC team analyzes the incident and improves security policies to prevent similar attacks in the future.

Security Information and Event Management (SIEM)

SIEM stands for Security Information and Event Management. It is a security solution that collects, analyzes, and correlates logs from different systems across the organization.

SIEM helps security teams monitor the network in real time and detect security threats quickly. It collects logs from servers, firewalls, routers, applications, and user devices.

By analyzing these logs, SIEM tools can identify suspicious activities such as unauthorized access attempts, malware infections, or unusual network traffic.

SIEM also helps organizations maintain compliance and provides detailed reports for security analysis.

SIEM Cycle

The SIEM system works through several stages.

Data Collection

Logs and events are collected from different devices such as servers, firewalls, routers, and applications.

Data Normalization

The collected logs are converted into a standard format so they can be analyzed easily.

Data Correlation

The SIEM system compares different logs and events to identify patterns that may indicate a security threat.

Alert Generation

If suspicious activity is detected, alerts are generated and sent to the SOC team.

Investigation

Security analysts investigate the alerts and determine whether the threat is real.

Response

If an attack is confirmed, the SOC team takes action to stop and mitigate the threat.

MISP (Malware Information Sharing Platform)

MISP is an open-source threat intelligence platform used to share information about cyber threats and indicators of compromise (IOCs).

It helps organizations collect and share information about malicious IP addresses, domains, file hashes, and malware attacks.

Security teams use MISP to improve threat detection and prevent cyber attacks. It allows different organizations to collaborate and share threat intelligence data to strengthen cybersecurity defenses.

College Network Information

In a typical college network, multiple systems and devices are connected to provide internet access and internal services for students, faculty, and staff.

The college network usually includes:

- Student computers and laptops
- Faculty systems
- Administrative servers
- Library systems
- Learning management systems
- Database servers
- Campus Wi-Fi networks
- Internet gateway and firewall

These systems generate large amounts of log data which can be monitored using SIEM tools to detect security threats.

Deploying SOC in a College Network

To deploy a SOC in a college environment, the following steps can be followed:

Network Monitoring

Install monitoring tools to observe all network traffic and system activities.

Log Collection

Collect logs from firewalls, servers, applications, and endpoint devices.

SIEM Implementation

Use a SIEM platform like IBM QRadar to analyze collected logs and detect security threats.

Threat Intelligence Integration

Integrate threat intelligence feeds such as MISP to detect known malicious IPs and domains.

Incident Response Process

Create a structured process for responding to security incidents.

Continuous Monitoring

The SOC team should monitor the network continuously to detect and respond to threats.

Threat Intelligence

Threat intelligence refers to the collection and analysis of information about potential cyber threats.

It helps security teams understand how attackers operate and identify possible threats before they occur.

Threat intelligence includes information such as:

- Malicious IP addresses
- Suspicious domains
- Malware signatures
- Attack techniques used by hackers

Using threat intelligence helps organizations improve their security defenses and respond faster to attacks.

Incident Response

Incident response is the process used to identify, manage, and recover from cybersecurity incidents.

The main stages of incident response include:

Preparation

Develop security policies and prepare tools to handle cyber incidents.

Detection

Identify suspicious activities and security alerts.

Containment

Isolate affected systems to prevent the attack from spreading.

Eradication

Remove malware or malicious activity from the system.

Recovery

Restore systems and return them to normal operation.

Post-Incident Review

Analyze the incident to improve future security measures.

QRadar – Understanding the Tool

IBM QRadar is a powerful SIEM platform used for security monitoring and threat detection.

QRadar collects logs from multiple sources such as servers, firewalls, applications, and network devices. It analyzes these logs and identifies suspicious activities.

Some important features of QRadar include:

- Real-time threat detection
- Log management and analysis
- Event correlation
- Security dashboards
- Incident investigation tools
- Threat intelligence integration

QRadar dashboards provide a visual view of security events, helping SOC analysts quickly understand and respond to threats.

Conclusion

Stage 1 – Web Application Testing

From Stage 1, we understood the importance of web application testing in identifying security vulnerabilities in web applications. Web application testing helps detect issues such as SQL injection, cross-site scripting, and authentication weaknesses. This testing ensures that web applications are secure and reliable for users.

Stage 2 – Nessus Report

From Stage 2, we learned how vulnerability scanning tools like Nessus identify security weaknesses in systems and networks. Nessus reports provide detailed information about vulnerabilities, their severity levels, and recommended solutions. This helps organizations fix security issues before attackers exploit them.

Stage 3 – SOC / SIEM / QRadar Dashboard

From Stage 3, we understood how SOC teams use SIEM platforms like

QRadar to monitor networks, detect threats, and respond to incidents. The QRadar dashboard provides real-time visibility into security events, helping analysts quickly identify suspicious activities and protect the organization's systems.

Future Scope

Stage 1 – Future of Web Application Testing

The future of web application testing will involve greater use of artificial intelligence and automation. AI-powered tools will be able to detect vulnerabilities faster and more accurately. Cloud-based testing platforms will also become more common, allowing organizations to test applications more efficiently.

Stage 2 – Future of Testing Process

The testing process will increasingly focus on automation and continuous testing. AI and machine learning technologies will help generate test cases automatically and detect complex security issues. This will improve software quality and reduce the time required for testing.

Stage 3 – Future of SOC / SIEM

In the future, SOC and SIEM systems will become more intelligent through the use of artificial intelligence and machine learning. These technologies will help detect threats automatically, analyze user behavior, and predict cyber attacks before they occur. Automation will also improve incident response and reduce the workload of security analysts.

Topics Explored

- Security Operations Center (SOC)
- SOC Cycle
- Security Information and Event Management (SIEM)
- SIEM Cycle
- Threat Intelligence
- MISP Platform
- Incident Response Process
- IBM QRadar Dashboard
- Web Application Testing

- Vulnerability Assessment using Nessus

Tools Explored

- Nmap
- Metasploit
- Nessus
- Kali Linux
- OWASP ZAP
- Burp Suite
- IBM QRadar